

GELB = checken mit L, Helena

GRÜN = erledigt

BLAU = eigene Fragen

ROSA: alleine nochmal durchgehen oder mit Lu

Woche 1:

- **Aufgabe 1:** Arbeiten Sie das Notebook *Hausaufgaben und Labs > Introduction to Python* durch (auch verlinkt am Ende vom Kapitel *Einführung in Data Science*). Beachten Sie die Aufgaben am Ende des Notebooks und versuchen Sie möglichst viele davon zu lösen. Lösungen mit ChatGPT, Claude etc. sind nicht erwünscht - sie bringen Ihnen zum jetzigen Zeitpunkt gar nichts.

Excercise 1 bis 5 durchgehen und checken ob Antworten richtig sind. Lösungen sind in „HA1_pythonbasics(2).ipynb

Woche 2:

- **Aufgabe 1** (Ende des Abschnitts "Palmer Penguins Datensatz"): Machen Sie sich mit der Forschung hinter dem Palmer Penguins Datensatz vertraut. Recherchieren Sie, welcher Zweck hinter der Datenerhebung steckt und welche Annahmen mit dem Datensatz verknüpft sind. Machen Sie sich auf diese Art mit dem Datensatz vertraut. Dokumentieren Sie dabei Ihre Fragen und Antworten.

Antwort: Den Datensatz in Pycharm in der Datei „penguinDatensatzAnschauen“ anschauen.

Eigene Fragen hier:

1a) Mit welchen Python Befehlen kann ich mir den Datensatz ansehen?

A: Siehe auch pedestrian_forecast_datascience Explorative Datenanalyse

A: „import pandas as pd“ importieren und mit „pd.read_csv(Pfad)“ kann ich mir den Datensatz anschauen.

„df.head()“ -> ersten 5 Zeilen

df.shape -> alle Zeilen und Spalten

df.describe() -> statistische Übersicht (nicht für nan und Strings)

```
df.isna().sum()
```

-> alle missing values werden angezeigt. Wenn 0 steht dann ist das gut weil bedeutet kein missing value.

```
df.isna().mean()
```

-> Prozent der fehlenden Werte pro Spalte

```
bike = df[["segment_id", "date_local", "uptime", "bike_lft",  
"bike_total"]].copy()
```

-> hier die Daten kopiert die ich haben will und damit weiterarbeiten um df nicht zu verändern. Ich habe hier hinter df eine Klammer [] um Spalten auszuwählen und eine [] um eine Liste hinzuzufügen.

```
bike #hiermit gibts auch ne Übersicht
```

```
bike = bike.drop(columns= ["segment_id", "uptime"])# kann Spalten und Zeilen  
löschen, deswegen muss explizit gesagt werden das Spalten gelöscht werden. Mit  
dem bike = wird es gespeichert
```

```
#Maximum von date_local Spalte
```

```
bike["date_local"].max()
```

```
#Min von date_local Spalte
```

```
bike[,"date_local"].min()
```

```
#Plot
```

```
bike["bike_lft"].plot()
```

```
bike.boxplot(column="bike_lft") # hiermit ein boxplot erstellt.
```

```
#Graph
```

```
#Datum als xAchse festlegen:
```

```
bike.set_index("date_local")["bike_lft"].plot() #set_index heißt das  
"date_local" auf die x achse kommt in verbindung mit der plot methode.
```

```
#Ergebnis nicht so gut weil die Zeitspanne so groß ist
```

```
bike.dtypes # dtypes zeigt die Datentypen der Spalten an
```

```
#Bei date_local steht object und das heißt das es wahrscheinlich ein String ist.
```

String ist.

#Wollen Date Local in ein Daytime Objekt umwandeln

pd.to_datetime(bike["date_local"]) #die methode to_datetime() wandelt Strings in Datetime Objekte um.

bike["date_local"] = pd.to_datetime(bike["date_local"]) #Wir weisen der Spalte jz Datetime Objekte statt Strings zu.

bike["date_local"].dt.year #an bike rufen wir die date_local Spalte auf und dann dt.year um das Jahr anzuzeigen. Für jedes Datum gibt es jetzt das Jahr aus.

das Jahr aus.

##%

bike["year"] = bike["date_local"].dt.year #hiermit eine Spalte erstellt die ich year nenne

bike

#

bike["month"] = bike["date_local"].dt.month

bike["week"] = bike["date_local"].dt.isocalendar().week

eine_woche = bike[(bike["year"]==2015) & (bike["week"]==30)] #filtern

eine_woche.set_index("date_local")["bike_lft"].plot()

2) Was ist zu beachten beim Pfad damit der Datensatz angezeigt wird?

A: Am besten mit Shell Befehl den Pfad checken.

!pwd #mit ausrufezeichen ist ein shell befehl, wo ich mein Pfad check

df = pd.read_csv('../data/bzm_ecocounter_100033037.csv') #pd.read_csv() gibt ein DataFrame zurück

3) Worin wird der Datensatz gespeichert?

-In df mit einer Zuweisung

4) Welche Librarys brauche ich?

5) Was macht pandas? A: Pandas ist für Datenanalyse, verarbeitung und manipulation. Ideal für Exploration.

6) Was macht matplotlib? A: Ist für Plots und statistische Visualisierungen wie Histogrammen, Diagrammen, Liniendiagrammen etc.

7) Wie lasse ich mir die ersten 3 Zeilen anzeigen aus einem Datensatz?

A: `df.head(3)`

8) Wie lasse ich mir alle Zeilen und Spalten anzeigen?

A: Am besten nicht alle anzeigen, aber ein sample ist gut:

```
df.sample(n=10)
```

oder für einen Bruchteil des Datensatzes (z. B. 25%):

```
df.sample(frac=0.25)
```

Mit `random_state=42` wird das Ergebnis reproduzierbar.

```
sample = df.sample(n=3, random_state=42)
```

9) Wie prüfe ich auf fehlende Werte?

A: `df.isna().sum()`

10) Wie kann ich die Python Pandas Library nutzen um mich mit der Verarbeitung tabellarischer Daten vertraut zu machen?

11) Wie führe ich erste Schritte zur Datenbeschreibung, Datenvisualisierung und Exploration durch?

A: Datenbeschreiben:

-`df.shape`

-`df.head`

-`df.describe()`

Datenvisualisierung:

```
-print(df.groupby('island')['species'].value_counts())
```

Exploration:

-mit Matplotlib Korrelationen ansehen

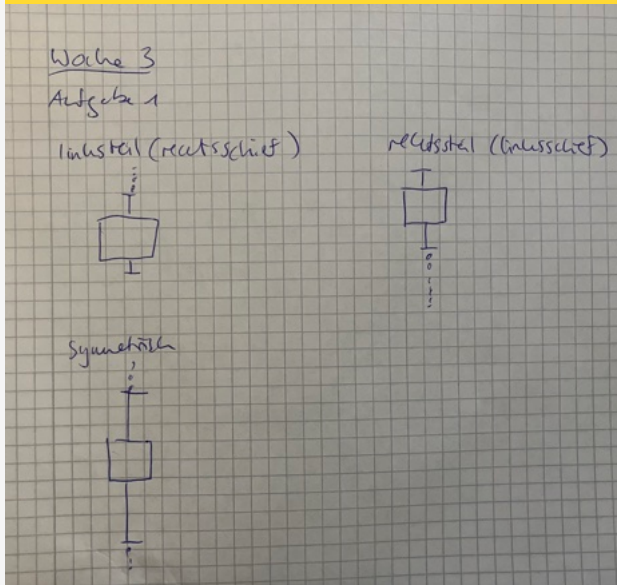
- **Aufgabe 2:** Arbeiten Sie das Notebook "Teil 2 der Einführung in Python" durch.

Excercise 1-3 durchgehen und Lösungen checken ob richtig. Die Lösungen sind in „HA2_python_functions_dicts.ipynb“.

Woche 3:

- **Aufgabe 1** (Ende vom Abschnitt "Deskriptive und explorative Datenanalyse"): Skizzieren Sie einen Boxplot, bei dem die zugrundeliegenden Daten linkssteil / rechtssteil / symmetrisch sind.

Antwort



Bedeutung /

Eigene Frage: Wie sieht der Python Befehl dafür aus?

```
bike.boxplot(column="bike_lft") # hiermit ein boxplot erstellt. Die Verteilung ist sehr schief und unsymmetrisch.
```

```
#Es gibt viele Ausreißer.
```

- **Aufgabe 2** (Ende vom Abschnitt "Data Preprocessing"): Nutzen Sie das Notebook "Python Pandas Library", um sich mit der Python-Bibliothek `pandas` für Verarbeitung tabellarischer Daten vertraut zu machen.

Eigene Fragen:

1) Was gehört zur Verarbeitung tabellarischer Daten und wie mache ich das mit pandas?

A: Siehe Visualisierung, Exploration etc.

2) Was gehört zum Data PreProcessing und wie mache ich das?

Nullwerte entfernen, Spalten mit Nullwerten, Spalten für z.B. für Datumsteile anlegen, String in Datetime umwandeln, Dataphrame Speichern, Plotten, Daten angucken, Überflüssige Spalten entfernen, Fehler anschauen wie z.B. female/male Verwechslung oder wenn etwas nicht gehen kann wie eine Plz die es nicht geben kann oder ein Alter über 100 ist, Plausibilität der Daten überprüfen, Encoding (One-Hot Encoding, Label Encoding), Normalisieren (damit die Daten vergleichbar sind)

3) Wie kann ich meine Antworten von „Hausaufgabe3“ dafür nutzen

- **Aufgabe 3** (Ende vom Abschnitt "Data Preprocessing"): Nutzen Sie die `pandas` Bibliothek, um sich mit dem Palmer Penguins Datensatz vertraut zu machen. Führen Sie erste Schritte der
-Datenbeschreibung,
-visualisierung und
Exploration durch.
Überprüfen Sie den Datensatz auf Fehler und fehlende Werte.

Lösungen in „Hausaufgabe3“ und `pedestrian_forecast_datascience` (Explorative Datenanalyse)

Eigene Fragen:

- 1) Was gehört zur (Datenbeschreibung) mit `pandas` für den Penguin Datensatz und wie mach ich das?
A: -Describe
- 2) Was gehört zur (Visualisierung) mit `pandas` für den Penguin Datensatz und wie mach ich das?
A: Scatterplot, Boxplot, Histogramm
- 3) Was gehört zur (Exploration) mit `pandas` für den Penguin Datensatz und wie mach ich das?
A: Gruppierungen (Bining?), Value-Counts und Korrelationen für Musterentdeckung.
- 4) Was gehört zur (Fehlerüberprüfung und fehlende Werte) mit `pandas` für den Penguin Datensatz und wie mach ich das?

- **Aufgabe 1:** Holen Sie die Hausaufgaben der ersten drei Wochen auf.
- **Aufgabe 2:** Aufgabe 1 am Ende des Abschnitts "Überwachtes Lernen" (so weit wie möglich)

Aufgabe 1 - Begriffe

Welche der folgenden Aussagen ist korrekt?

1. Trainingsdaten werden verwendet, um ein überwachtes maschinelles Lernmodell zu trainieren.
2. Ein Modell ist eine spezifische Instanz einer Modell-Familie.
3. Ein Lernalgorithmus wird verwendet, um das optimale Modell einer Modell-Familie zu finden
4. Überwachtes Lernen ist eine Form des maschinellen Lernens, bei der das Modell aus gelabelten Daten lernt.
5. Eine Überanpassung/Overfitting liegt vor, wenn ein Modell zu einfach ist und die Komplexität der Daten nicht erfassen kann.
6. Ein Feature ist eine messbare Eigenschaft oder Charakteristik der Daten.
7. Die Testdaten werden zum Trainieren eines maschinellen Lernmodells verwendet.

A:

1. Ja
2. Ja, bsp. KNN ist eine Modellfamilie
3. Ja Das ist das Verfahren der das optimale Modell findet. Bsp KNN.
4. Ja
5. Nein
6. Ja
7. Nein, die Testdaten werden zum überprüfen der Modelleistung und nicht zum trainieren verwendet.

[Lösungen eingefügt.](#) (hausaufgabe 10.11.25 auf tablett)

Aufgabe 2 - Wahl der Fehlermetrik

In der Vorlesung haben wir den Mean Squared Error (MSE) als die übliche loss function für Regression kennengelernt. Angenommen Sie wollen die optimale lineare Funktion zu dem Datensatz in der folgenden Abbildung bzgl. MSE finden.



1. Wie könnte diese in etwa aussehen, wenn Sie die Ausreisser/Outlier unten rechts ignorieren? Zeichnen Sie Ihre Lösung ein.
2. Was ändert sich, wenn Sie die Ausreisser mit berücksichtigen?
3. Wie könnten Sie die Formel des MSE anpassen, um diesem Effekt etwas entgegen zu wirken?

Lösungen einfügen!

A:1. Die Gerade ginge durch die Punktwolke

A:2: Die Gerade wäre nach unten verschoben (FRAGE HELENA ginge es vom NULLpunkt aus und wäre gekippt?)

A:3: Huber Loss wäre eine Möglichkeit, ist robust gegen Outlier und die Fehler sind teilweise linear oder quadratisch. Je nach dem ob es ein Ausreiser ist.

- statt es quadratisch zu gewichten, kann man nur den mean error nehmen (also den Betrag der Abweichung) und das nennt sich mittlere absolute Abweichung MAD. Große Abweichungen fallen nicht so sehr ins Gewicht.

(FRAGE HELENA, welches Verfahren am besten, oder wie am besten damit umgehen?)

- **Aufgabe 1:** Fortsetzung: Aufgabe 1 am Ende des Abschnitts "Überwachtes Lernen"
- **Aufgabe 2:** Aufgabe 2 am Ende des Abschnitts "Überwachtes Lernen"

- Aufgaben 1-3 vom Kapitel „k-NN“

Aufgabe 1 - Algorithmus-Logik

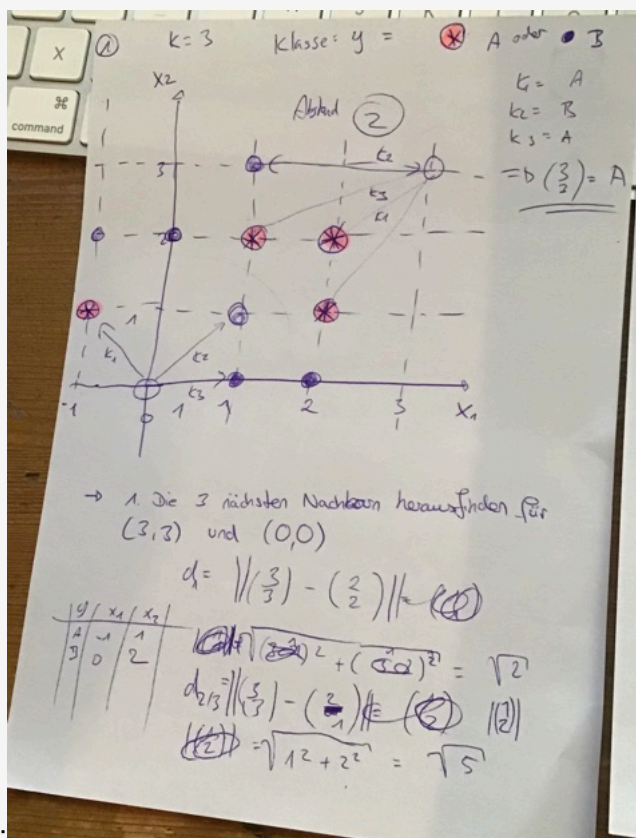
Gegeben ist folgender Trainingsdatensatz für einen 3-NN-Klassifikator:

```
import pandas as pd

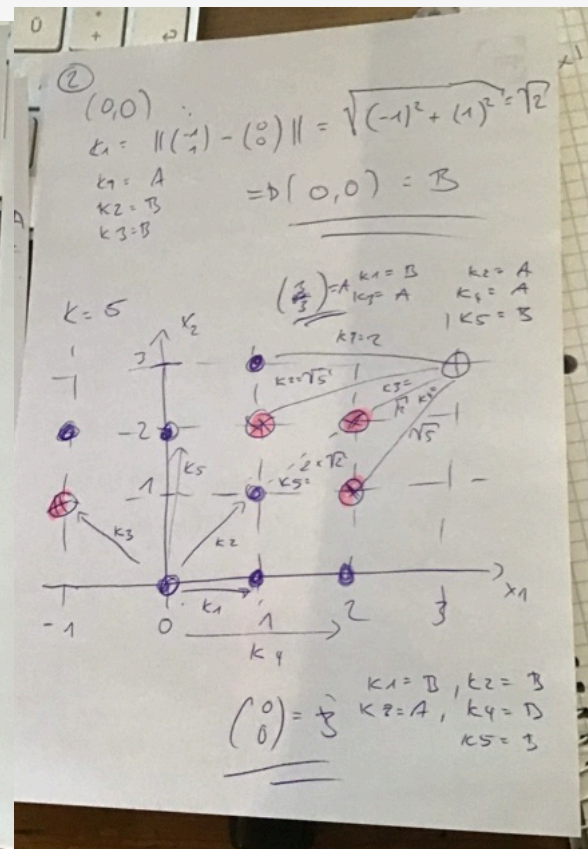
data = pd.DataFrame([
    [-1, 1, 'A'], [0, 2, 'B'], [1, 2, 'A'], [2, 1, 'A'],
    [-1, 2, 'B'], [1, 1, 'B'], [1, 3, 'B'], [1, 0, 'B'],
    [2, 0, 'B'], [2, 2, 'A']
], columns=['x1', 'x2', 'y'])
```

- Ordnen Sie die Punkte (3,3) und (0,0) einer Klasse zu ($k=3$).
- Was ändert sich bei $k = 5$?
- Fertigen Sie für beide Fälle Skizzen an.

A: siehe Block (von mir geschrieben, muss noch überprüft werden)



A:



Aufgabe 2 — Rechenaufwand ableiten

Angenommen, unsere Daten sind d-dimensional. Wir betrachten k-NN so, wie in der Vorlesung beschrieben, d. h.:

1. Für jeden der Trainingspunkte den Abstand zur Query (neuer Datenpunkt) berechnen.

2. Aus allen Abständen die k-kleinsten bestimmen (z. B. vollständig sortieren oder nur „top-k“ auswählen).

3. Klassifikation: Mehrheitsvotum der k-Nachbarn; Regression: Mittel/Median der k-Zielwerte.

- a) Welche Teiloperation bestimmt typischerweise den größten Rechenaufwand pro Query – Schritt 1 oder 2? Begründe.
- b) Wie ändert sich der Gesamtaufwand, wenn m Queries bearbeitet werden (Batch)?

- SIEHE BILD laufzeit Ordner L
-

(Antwort KI generiert)

a) Antwort1:

Den größten Rechenaufwand pro Query verursacht typischerweise Schritt 1 (Abstände zum neuen Datenpunkt berechnen)

Begründung: Schritt 1 berechnet n Abstände, je $O(d)$ Arbeit $\rightarrow O(n \cdot d)$.

Schritt 2 sortiert/extrahiert Top-k $\rightarrow O(n \log k)$ (Heap) oder $O(n \log n)$.

Bei typischem $k \ll n$ ($k=5$, $n=10^5$) dominiert $O(n \cdot d)$.

- a) Antwort2:
- Gegeben Trainingsdatensatz mit n Punkten
- Dimension d

Schritt 1: Abstände berechnen.

Für jeden der n Trainingspunkte wird der Abstand zur Query berechnet.

- Typisch: euklidischer Abstand

- Aufwand pro Abstand $O(d)$

- Gesamtaufwand Schritt 1: $O(n \cdot d)$

Schritt 2: kleinste Abstände bestimmen

Aus n Abständen werden die k-kleinsten ausgewählt.

Mögliche Verfahren:

- Vollständiges Sortieren $O(n \log n)$

- Top-k mit Heap $O(n \log k)$

In der Praxis gilt meist d ist groß z.B. 10-1000

$k \ll n$

Abstandsberechnungen bestimmen typischerweise den größten Rechenaufwand für jeden Trainingspunkt, da für jeden Trainingspunkt ein d-dimensionaler Abstand berechnet werden muss.

b) Antwort1: Gesamtaufwand $O(m \cdot n \cdot d)$ – linear skalierend mit m. Praktisch effizienter durch vektorisierte Distanzmatrix-Berechnung (NumPy/TensorFlow).

b) Antwort2: Batch mit m Queries: Aufwand wächst linear in m.
 $O(m \cdot n \cdot d)$

Aufgabe 3 - Implementierung

Erweitern Sie eines der k-NN Modelle aus dem Unterricht, indem Sie weitere Variablen miteinbeziehen oder unterschiedliche Werte von k ausprobieren.

A: Siehe Woche 7 Aufgabe 4
nochmal üben!

eigene Fragen:

1) Was sind Hyperparameter?

A: Das sind die Parameter die **vor dem eigentlichen Training** festgelegt werden.

Bei k-NN das k.

2) Wie kann ich den Einfluss von Hyperparametern messen?

A: Das Modell mit verschiedenen Hyperparametern trainieren und die Performance auf den Validierungsdaten vergleichen.

3) Wie kann ich die optimalen Hyperparameter-Werte finden?

A: bsp. durch die Cross Validation, Grid Search

4) Wenn das k zu klein ist dann habe ich einen kleinen Fehler bei den Trainingsdaten und einen großen bei den Testdaten, wieso?

A: Wenn das k klein ist (z.B. $k=1$ bei KNN) dann schaut das Modell nur auf die nächsten 1 oder 2 Trainingspunkte, um die Vorhersage zu treffen. Das Modell passt sich sehr gut an die Trainingsdaten an und die Trainingsdaten werden fast perfekt vorhergesagt. -> kleiner Fehler auf Trainingsdaten.

Aber weil sich das Modell jedes Rauschen der Trainingsdaten lernt, überträgt sich das Rauschen nicht gut auf neue Daten. Und neue Testdaten können völlig anders aussehen -> großer Fehler auf den Testdaten.

(Bei kleinem k hat k-NN eine hohe Varianz und neigt zum Overfitten. Es passt sich stark an einzelne Trainingspunkte und Rauschen an, wodurch der Trainingsfehler klein, der Testfehler jedoch groß wird.)

5) Wenn das k groß ist dann wächst bzw. stagniert der Testfehler und Trainingsfehler (Frage was genau passiert?) und wieso?

Was macht die Entscheidungsgrenze?

6) Steht das k für den Hyperparameter?

Bei knn ja.

7) Was ist der Effekt von Hyperparametern auf die Performance vom Modell?

A: Hyperparameter steuern die Modellkomplexität. Eine schlechte Wahl führt zu Underfittung/Overfitting. Und eine gute Wahl verbessert die Generalisierung.

8) Wie hängt Overfitting und Underfitting mit der Modellkomplexität zusammen?

A: Hohe Modell Komplexität bedeutet Overfitting. Trainingsfehler sehr klein, Testfehler sehr hoch.

Niedrige Modell Komplexität bedeutet Underfitting. Trainingsfehler sehr hoch und Testfehler sehr hoch. Das Modell ist sehr dünn.

9a) Was ist der Unterschied zwischen Bias und Varianz?

A: Die Art des Fehlers. Der Bias ist eine Art Voreingenommenheit des Modells. Es hat eine bestimmte Vortendenz. Die Varianz

(Es gibt den irreducible Error: Der Fehler den ich nicht wegbekomme. Gibt immer eine Art an Rauschen die ich nicht erklären kann. Bsp Wetter, weil Granularität zu hoch die ich bräuchte)

Modelle die underfitten machen in der Regel ein Bias Fehler.

Modelle die overfitten machen in der Regel hohen Varianzfehler.

Bsp. Daten gesammelt Restaurant, Rechnungsbetrag und Trinkgeld erhoben. Modell trainiert. Dann Fehler berechnet. Dann wiederhole ich das und gehe an andere Tische hole Trinkgeld etc. trainiere erneut und werte auf einen Test Datensatz aus. Und mache das noch mehrmals. Andere Tage und Tische. Jedes mal etwas verschiedene Daten.

Deswegen leicht verschiedene Modelle. Jetzt wollen wir einen neuen Datenpunkt ist ne Rechnung gekommen. Kellner machen Prediction wieviel Trinkgeld wer gibt als Werte. Und dann kommen verschiedene Vorhersagen raus. Der Fehler den man durch Bias bekommt, ist die Differenz zwischen der Durchschnittlichen Vorhersage und den wahren Zielwert.

Haben immer wieder Daten gesammelt in einer bestimmten Woche und Modelle trainiert. Ich bilde von den allen den Durchschnitt. Und der Durchschnitt ist 2,5 euro. Aber Person sagt 5. Dann ist hier ein Bias drin. Weil alle liegen irgendwie zu weit weg. Weil das Modell unterschätzt systematisch.

Bei hoher Varianz streuen die Vorhersagen sehr stark.

9b)

Angenommen, Sie wollen vorhersagen, wie viel Prozent der potenziellen Wähler*innen für eine Randbebauung des Tempelhofer Feldes stimmen werden. Sie entscheiden sich für folgendes Vorgehen:

1. Wähle zufällig 50 Nummern aus dem Telefonbuch
2. Rufe jede Nummer an und Frage, wie die Person beabsichtigt abzustimmen

Sie erhalten folgendes Ergebnis:

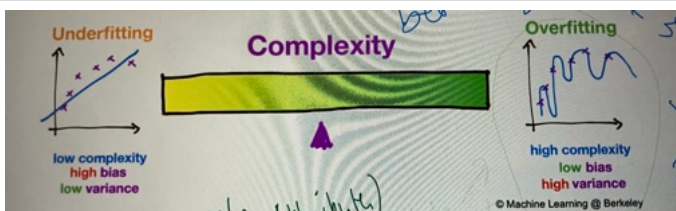
Dafür	Dagegen	Keine Antwort	Gesamt
13	16	21	50

Die geschätzte Wahrscheinlichkeit, dass für die Bebauung gestimmt wird, liegt bei ca. 0,45.

- Welche Fehlerquellen sehen Sie bei diesem Vorgehen?
- Können Sie bestimmen, ob die Fehlerquelle eher den Bias oder die Varianz erhöht?

Einige Fehlerquellen

- Auswahl aus Telefonbuch: bezieht nur Personen mit ein, deren Nummer gelistet ist (Bias)
- Die nicht-Antwortenden auszulassen führt zu einer konsistenten Verzerrtheit der Daten (Bias)
- Wenige Daten (nur 50) führt zu hoher Varianz



10) Sind Validierungsdaten das gleiche wie Testdaten?

A: Nein. Validierungsdaten werden während der Modellentwicklung genutzt. Sie dienen zur Auswahl der Hyperparameter (z.b. bestes k bei kNN). Testdaten hingegen werden erst ganz am Ende verwendet. Sie sind für die

finale Bewertung des Modells. Sie dürfen nicht zur Modellauswahl oder Hyperparameterauswahl genutzt werden. Sie simulieren neue, unbekannte Daten.

11) Was ist die Cross-Validation?

A: Mit der cross-Validation wird ein Datensatz mehrfach in Trainingsdaten und Validierungsdaten aufgeteilt. Ziel ist die Modelleistung stabil zu schätzen und die optimalen Hyperparameter herauszufinden. Also bei KNN das beste k.

Vorgehen:

1. Ich wähle mehrere Kandidaten für k. z.b. $k=1,3,5,7$
2. Für jedes k führe ich die Cross Validation durch und berechne den durchschnittlichen Validierungsfehler
3. Ich wähle das k mit den kleinsten durchschnittlichen Fehler -> Das ist der optimale Hyperparameter.

Genauer: Wenn die Menge D der gelabelten Daten gering ist dann ist ein einfacher Split in Dtrain und Dval nicht zuverlässig genug. In dem Fall wird eine Cross Validierung durchgeführt.

Dabei teilen wir die Menge D/ Dtest mehrfach in Dtrain und Dval (gelb) auf.

Auf jedem der sog. **Training Folds** trainieren wir Modelle für die verschiedenen Werte von k (oder anderer Hyperparameter) und berechnen die Loss auf dem jeweiligen **Validation Fold**

Diejenige Wahl von Hyperparameter, die den geringsten durchschnittlichen Val-Fehler hat, ist optimal Beispiel (siehe Abbildung): Wähle diejenigen Hyperparameter, die den mittleren Validation-Error minimieren

11b) Was ist der Unterschied zur Nested Cross Validation?

12) Wie setzte ich Validierungsdaten für die Cross-Validation ein?

13) Was sind nochmal die gelabelten Daten D? (in Bezug auf Modellkomplexität VL8)

A: Das sind Trainingsdaten die aus Eingaben und zugehörigen Zielwerten bestehen. Formal $D = \{(x_1, y_1), (x_2, y_2), \dots, (x_n, y_n)\}$

x_i = Input / Features

y_i = Output / Label (Klasse oder Zielwert)

Gelabelt heißt wir wissen schon was das richtige Ergebnis ist, im Gegensatz zu unüberwachten Daten. Ein Bsp. ist Spam.

14) Erkläre die Grafik:

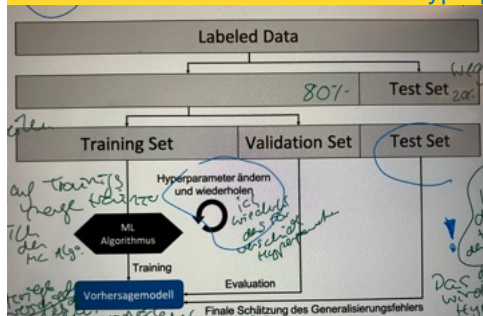
Gelabelte Daten werden in 80% Training und 20% Test Set geteilt.

Training wird nochmal in Training Set 80% und Validation Set 20% geteilt.

Auf Trainingsmenge trainiere ich den ML Algo. Ich wiederhole das für verschiedene Hyperparameter.

Ich kriege verschieden Vorhersageodelle und vergleiche die mit der Validierungsmenge.

Das beste Modell mit den besten Hyperparametern wird dann nochmal im Test Set ausgewertet.



Siehe Notizen VI8 iPad

13) Was ist die capacity? A: (Kapazität) Mit wachsendem k fällt die Kapazität des Modells beliebige Muster zu lernen.

14) Frage aus dem Skript: Was passiert, wenn gleich der Anzahl an Punkten in der Trainingsmenge ist?

A: Maximal viele Nachbarn: Jeder neue Punkt hätte die gleichen Nachbarn, und zwar alle Punkte in der Trainingsmenge, und die Vorhersage wäre für alle gleich, nämlich die am häufigsten vorkommende Klasse in den Trainingsdaten. Ich sehe gar keine Muster mehr. Für alle gibt es das gleiche Muster.

15) Was ist die Modellkomplexität? A: Die Modellkomplexität beschreibt wie flexibel ein Modell ist. Also wie stark es sich an Details und Variabilität der Daten anpassen kann.

Bei k -NN:

kleines k -> sehr flexibel -> höhere Komplexität

größeres k -> weniger flexibel -> glattere Entscheidungsgrenze, weniger komplexes Modell

16) Was ist Underfitting?

A: Modell ist zu einfach -> Muster werden übersehen -> hohe Fehlerquote bei Trainings- und Testdaten

17) Was ist Overfitting?

A: Überanpassung: Modell zu komplex -> passt sich an Rauschen in den Trainingsdaten an -> geringer Trainingsfehler, aber hoher Testfehler.

18) Ist bei Overfitting das k eher zu klein? Und bei Underfitting das k eher zu groß?

A:

-Ja, Bei k -NN ist beim Overfitting das k eher zu klein (RICHTIG!)

-und beim Underfitting das k eher zu groß. (RICHTIG!)

19) Was ist der Unterschied zwischen Bias und Variance?

A: Underfitting -> Bias

Overfitting -> hohe Varianz

Fehler durch Bias: Differenz zwischen der durchschnittlichen Vorhersage aller Modelle und dem wahren Zielwert.

Fehler durch Varianz: Variabilität der Vorhersagen im Punkt.

20) Wenn ein Modell eine hohe Komplexität hat ist das k dann eher klein?

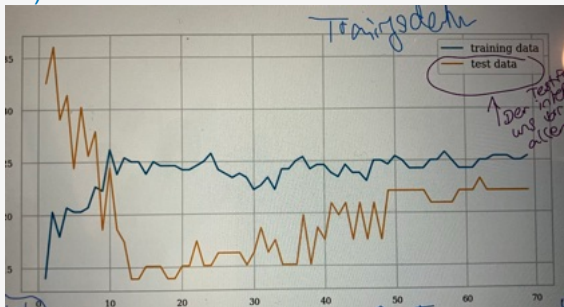
-Ja (das Modell ist empfindlich für Rauschen und „zerklüftete“ Entscheidungsgrenzen. Es gibt ein Overfitting-Risiko.

Bei einem großen k ; ist die Modellkomplexität gering. Es werden viele Nachbarn gemittelt (der Durchschnitt berechnet) und die Entscheidungsgrenze ist glatt. Das Modell ist wenig flexibel und es gibt Underfitting-Risiko.



siehe Erklärung/Bild ipad VL8

21) Erkläre die Grafik:



Siehe Grafik/Erklärung ipad VL8

21) Was sind Freiheitsgrade? A: das sind die Parameter die ein Modell lernen muss um sich an die Trainingsdaten anzupassen. Bei einem Polynom mit Grad 5 z.B. eher flexibel mit hoch und Tiefpunkten. Bei Polynom 1. Grades, weniger Parameter) ist es eine Gerade und kann nur allgemeine Trends abbilden.

- Aufgabe 1,2,4 vom Kapitel „Modellauswahl“

Aufgabe 1 - Konzepte unterscheiden

Erklären Sie den Unterschied zwischen Trainingsdaten, Validierungsdaten und Testdaten, und deren jeweilige Rolle im Modellierungsprozess.

L Lösung:

Trainingsdaten:

Werden verwendet um verschiedene Modelle aus verschiedenen Modellfamilien und mit verschiedenen Hyperparametern zu trainieren. Anpassung der Modellparameter.

Validierungsdaten: Werden verwendet um die trainierten Modelle miteinander zu vergleichen und zu entscheiden, welches Modell das beste ist. Auswahl der Hyperparameter.

Testdaten: Werden dazu verwendet das Modell mit den Testdaten (nicht mit den Trainingsdaten) laufen zu lassen und einen unbekannten „Punkt“ vorherzusagen. Gültigkeit vom Modell checken.

—

Aufgabe 2 - Modellauswahl verstehen

Sie testen verschiedene Werte von k für k -NN und erhalten folgende mittlere Fehlerwerte:

k	Trainingsfehler	Validierungsfehler
1	0.00	0.35
3	0.12	0.28
5	0.18	0.24
7	0.21	0.25
9	0.25	0.27

Fragen:

- Welchen Wert von k würden Sie für das endgültige Modell wählen?
- Warum steigt der Trainingsfehler mit zunehmendem k ?
- Warum zeigt der Validierungsfehler ein Minimum bei $k = 5$ und steigt danach wieder?
- Was würde passieren, wenn Sie sehr große Werte für k (z. B. $k = 50$) testen würden?

-Welchen Wert von k würden Sie für das endgültige Modell wählen?

A: L. Lösung: Welchen Wert von k würden Sie für das endgültige Modell wählen?

Ich würde $k=5$ wählen, weil da das Minimum für den Validierungsfehler ist.

-Warum steigt der Trainingsfehler mit zunehmendem k ?

A: Der Trainingsfehler steigt mit zunehmenden k , weil das Modell weniger flexibel wird und sich schlechter an Trainingsdaten anpasst. (Underfitting)

Es wird dann irgendwann einfach eine Mehrheitsentscheidung

Antwort gehört an eine andere Stelle

L A: Es ist eine generalisierende Aussage, wenn z.B. ein Punkt mit allen anderen Punkten erklärt wird. Bei $K=1$ wäre der Fehler $=0$ auf den Trainingsdaten, weil man einen Punkt des Datensatzes ausgewählt und der sich selbst der nächste Nachbar ist.

-Warum zeigt der Validierungsfehler ein Minimum bei $k=5$ und steigt danach wieder?

A: Weil er bei einem kleinen k das Modell überanpasst (overfitting) und bei großem k unteranpasst (underfitting)

-L Antwort: Der Fehler sinkt zunächst, weil mehr k die neuen Punkte besser beschreiben. Ab einem bestimmten k (hier $k=5$) haben die hinzukommenden Punkte keinen Mehrwert mehr zur Bestimmung des Punktes. Es wird weniger komplex- also komplexere Muster werden nicht mehr erkannt. (underfitting)

-Was würde passieren, wenn Sie sehr große Werte für (z. B. $k=50$) testen würden?

A: bei großen werten, z.b. $k=50$ wäre das modell so unflexibel dass es alle Punkte betrachtet und immer die am häufigsten vorkommende Klasse vorhersagt, was zu schlechter leistung führt.

-L A: Test- und Trainingsdaten haben beide einen recht hohen Fehler. → Mehrheitsklasse gewinnt

Aufgabe 3 - Bewertungs des Verfahrens

In einer Publikation steht: "Wir haben unsere Hyperparameter mit 5-fach Cross-Validation optimiert und anschließend den mittleren CV-Fehler als finale Performance berichtet." Ist das korrekt? Wenn nicht, warum nicht? Wie müsste man es richtig machen?

A: Es ist nicht korrekt, nur den mittleren Cross-Validation-Fehler als finale Performance zu berichten, da die Validierungsdaten indirekt zum Training genutzt wurden. Richtig wäre, nach Hyperparameterwahl das Modell auf allen Trainings- und Validierungsdaten zusammen neu zu trainieren und dann auf einem separaten Testdatensatz zu bewerten.

L A: Für jedes k wird ein Fehler berechnet $\rightarrow$ 5 mal

Das beste k ist das mit dem geringsten Fehler. Das wird ausgewählt. Mit Trainingsdaten + Validierungsdaten mit dem besten k wird ein neues Modell trainiert. Die finale Performance sollte auf den Testdaten ermittelt werden.

Aufgabe 4 - Berechnung und Darstellung des Trainings- und Validierungsfehlers

Führen Sie für eine andere Feature-Auswahl des Palmer-Pinguins Datensatzes einen Train-Val-Split mit 20 % Validierungsdaten und `random_state=1` durch. Trainieren Sie für k von 1 bis 30 ein k -NN Modell auf den Trainingsdaten und berechnen Sie die Ungenauigkeit jeweils auf beiden Mengen. Stellen Sie die Ergebnisse als Liniendiagramme in einer Abbildung dar. Sie können den Code aus der Vorlesung als Vorlage verwenden.

A: CODING-AUFGABE siehe L Lösung(ist aber nicht richtig)

Lösung(von mir ist richtig)

In Datascience, Penguin Aufgaben, Ha_Woche7_Modellkomplexität_TrainTestSplit

Aufgabe 5 - Cross-Validation

Führen Sie eine 5-fold Cross Validation durch statt eines einfachen Train-Val-Splits. Geben Sie die Genauigkeit auf allen Folds aus. Welcher Wert von k ist optimal?

A: Siehe Lösung: in DataScience25, im penguin Ordner; Ha_Woche7_Modellkomplexität_5Fold

Am Boxplot ist zu sehen das der optimale k Wert 3 ist.

überprüft aber am besten nochmal ÜBEN!

Lösung von L in „HA9.py“

[1] k -NN ist kein parametrisches Verfahren aber die Trainingsbeispiele haben eine ähnliche Funktion wie die Gewichte in einem polynomiellen Modell.

Eigene Frage: Was bedeutet das [1]?

A: Ein parametrisches Verfahren: haben eine feste Anzahl an Parametern, die beim Training gelernt werden.

KNN: hat keine festen Parameter

Eigene Fragen

1) Was mache ich mit nicht_numerischen Daten?

A: aus kategorischen Daten werden numerische Daten gemacht:

2 Möglichkeiten:

-Labelencoding (geht nur bei geordneten Größen wie z.b. tshirt größen) und

-One-Hot-Encoding (Wenn ich z.b. nur wissen will, lebt der Vogel auf der Insel: ja/nein).

-Bei Labelencoding: werden die Abstände berechnet und die Vektoren voneinander abgezogen.

-Bei OneHotEncoding nur 0 und 1 eingetragen. (Solange nicht zuviel Daten wie z.b. bei Amazon Produkte ist es kein Problem)

*Wenn Zeit weitere Infos ergänzen

Wofür ist der Validierungsfehler?

A: Der Validierungsfehler ist sowas wie eine Simulation & ich nutze es um die Hyperparameter auszuwählen.

*Wenn Zeit weitere Infos ergänzen

Woche 8: (VL9 Lineare Modelle für Regression)

- Aufgaben 3 und 5 vom Kapitel "Modellauswahl"
- Aufgaben 1 und 2 vom Kapitel "Lineare Modelle für Regression"

Aufgabe 1 - OR als Perzeptron

Stellen Sie die Wahrheitstabelle für die OR-Verknüpfung auf und zeigen Sie, dass OR als Perzeptron dargestellt werden kann.

A: Wahrheitstabelle OR:

x1	x2	y
0	0	0
0	1	1
1	0	1
1	1	1

Meine Lösung ist in roter Mappe, die Werte müssen richtig gewählt sein damit es geht.

L Lösung:

Aufgabe 1 - OR als Perzeptron

Stellen Sie die Wahrheitstabelle für die OR-Verknüpfung auf und zeigen Sie, dass OR als Perzeptron (Aktivierungsfunktion: wenn $w \cdot x + b \geq 0 \rightarrow 0$, sonst 1) dargestellt werden kann.

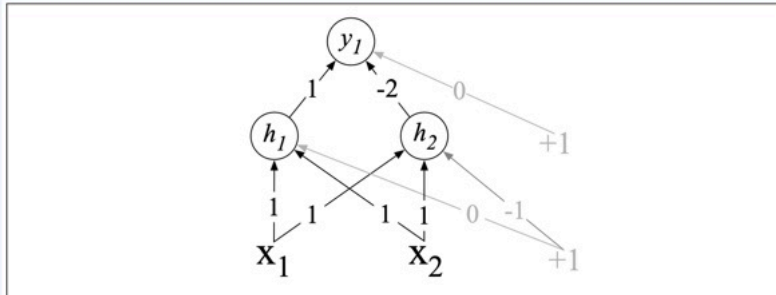
x1	x2	y	$w \cdot x + b$	$w \cdot x + b > 0$
0	0	0	0	0
0	1	1	1	1
1	0	1	1	1
1	1	1	2	1

Mit z.B. $b=0$ und $w=(1,1)$ möglich

WIESO ist da oben eine 2? Wenn Zeit dann nochmal überprüfen.

Aufgabe 2 - XOR als mehrschichtiges Neuronales Netz

Schon früh wurde erkannt, dass die Leistung neuronaler Netze, wie bei den echten Neuronen, aus der Kombination der Rechen-Einheiten zu größeren Netzwerken kommt. Eine der geschicktesten Demonstrationen der Notwendigkeit von mehrschichtigen Netzwerken war der Beweis von Minsky und Papert (1969), dass eine einzelne neuronale Einheit einige sehr einfache Funktionen wie XOR aus dem Input nicht berechnen kann. Prüfen Sie nach, dass folgendes Netzwerk mit einem hidden Layer und ReLU als Aktivierungsfunktion für die Inputs (0, 0) und (0, 1) den korrekten Output generiert.



A: muss ich noch machen: Lösung in roter Mappe

L A: Aufgabe 2 - XOR als mehrschichtiges Neuronales Netz

Schon früh wurde erkannt, dass die Leistung neuronaler Netze, wie bei den echten Neuronen, aus der Kombination der Rechen-Einheiten zu größeren Netzwerken kommt. Eine der geschicktesten Demonstrationen der Notwendigkeit von mehrschichtigen Netzwerken war der Beweis von Minsky und Papert (1969), dass eine einzelne neuronale Einheit einige sehr einfache Funktionen wie XOR aus dem Input nicht berechnen kann. Prüfen Sie nach, dass folgendes Netzwerk mit einem hidden Layer (man rechnet einmal um) und ReLU (eine nicht-Lineare Funktion?) als Aktivierungsfunktion für die Inputs (0,0) und (1,1) und den korrekten Output generiert.

y	x1	x2	h1	h2	Relu
0	0	0	0	0	0
1	0	1	1	0	1
1	1	0	0	1	1
0	1	1	2	1	0

Aufgabe 3 - Lineare regresison implementieren

Trainieren Sie eine Lineare Regression anhand des Palmer Penguins Datensatzes. Sie können die Features sowie die Zielvariable selbst bestimmen. Nutzen Sie hierfür die scikit-learn Klasse `sklearn.linear_model.LinearRegression`. Geben Sie sich anschließend die Gewichte und den Bias des trainierten Modells und interpretieren Sie das Ergebnis.

[Lösung von L in HA10_Lineare_regression.py](#)

[1] Lineare Aktivierungsfunktionen machen nicht viel Sinn, da die Hintereinanderausführung linearer Funktionen wieder eine lineare Funktion ist.

Eigene Fragen:

1) Erkläre die Grafik in Aufgabe2 - XOR als mehrschichtiges NN genauer.

2) Was ist ein hidden Layer?

A: Bei NN gibt es 1 Neuron pro Klasse. Und dazwischen gibt es beliebig viele Neuronen (Hidden Layers) wie ich haben will. Sobald ein Neuronales Netzwerk einen Hidden Layer hat heißt es Deep Neuronal Network. (Und Deep Learning ist das Training von **tiefen** neuronalen Netzen. In der Regel hat man viele hidden Layer.)

Ein Hidden Layer ist eine Schicht in einem neuronalen Netzwerk zwischen der Eingabeschicht (den Features) und der Ausgabeschicht (der Vorhersage). Jede der verborgenen Ebenen besteht aus einem oder mehreren Neuronen. Ein Hidden Layer transformiert die rohen Eingaben durch gewichtete Verbindungen und Aktivierungsfunktionen.

3) Was ist ReLU als Aktivierungsfunktion?

A: Siehe Antwort 4.

4a) Was genau ist eine Aktivierungsfunktion und wofür wird sie eingesetzt?

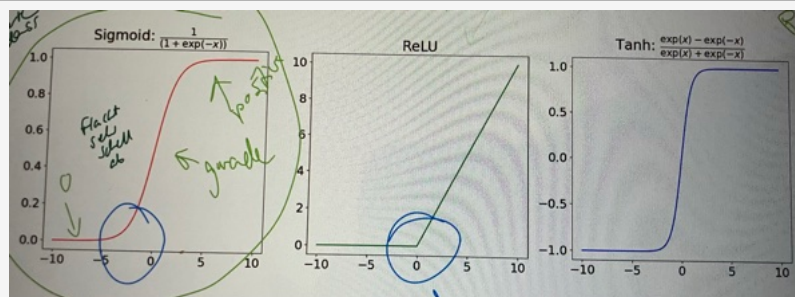
Es gibt viele Aktivierungsfunktion. Sigmoid, ReLU und Tanh Funktion gibt es. Siehe Bild.

Was haben die 3 gemeinsam? Wenn eine Zahl sehr negativ ist dann machts sie es zur 0. Und das machen die alle.

Relu, sagt alles was kleiner wird 0 und alles danach die Zahl selbst.

Tanh ist auf 1 begrenzt.

Also negative Werte sorgen dafür dass das Neuron nahezu nichts mehr nach außen gibt. Und hohe Werte führen dazu das es mit einer 1 feuert. -> Das ist was die Aktivierungsfunktion bewirkt.



4b) Was haben die 3 Aktivierungsfunktionen gemeinsam? Wenn ein sehr negativer Input kommt dann macht die Aktivierungsfunktion das mehr oder weniger zu einer 0. Und das machen die alle.

5) Was ist ein Perzepton?

A: Ein PERzepton ist eine sehr einfache Recheneinheit die binären Output 0 oder 1 produziert. Ein Perzepton hat eine bestimmte Art von Aktivierungsfunktion.

Wenn gewichtete Summe kleiner als 0 -> spuckt 0 aus.

Wenn größer als 0 dann 1.

6) Was für Architekturen von NN gibt es?

A: Bsp. Feed Forward, recurrent, conventional... die Wahl der Architektur hängt von Problem und der Daten ab.

7) Was ist ein Unit?

A: Ein Unit ist ein Neuron. Also ein grundlegender Baustein eines NN. Ein Unit nimmt einen Vektor reeller Zahlen als Eingabe, führt eine Berechnung damit durch und erzeugt eine Ausgabe. Für die Berechnung gibts 2 Schritte:

1. Schritt: berechnet die gewichtete Summe der Eingaben mit einer zusätzlichen additiven Konstante b, sog. Bias.

2. Schritt: Aktivierung einer nicht-linearen Funktion f, der sog. Aktivierungsfunktion. (Bsp. Sigmoid-Funktion)

8) Was hat es mit den Layers auf sich?

A: -Die Units in einem NN werden i.d.R. in Schichten (Layers) zusammengefasst.

-Input werte wandern von input schicht zu output schicht und können einzelne Schichten mehrfach durchlaufen.

Im einfachsten Fall laufen die inputs nur in eine Richtung und passieren jede Schicht nur einmal.

9) Was wird unter den Architekturen verstanden?

A: Gesamtstruktur/Layout des Netzes einschließlich der Anzahl und Anordnung der Schichten, Anzahl der Neuronen in jeder Schicht, die verwendete Aktivierungsfunktion und die Verbindungen zwischen den Neuronen

7) Im Skript die Übung mit der RechenUnit, ist 0,87 richtig als Ergebnis? Sollten den Wert schätzen oder am Graphen ablesen. Wäre das dann richtig?

8) Zeige das AND mithilfe eines Perceptrons berechnet werden kann.

✓ Lösung anzeigen

Eine mögliche Wahl ist z. B. $w = (1, 1)$ und $b = -1$:

- $(0, 0): w \cdot x + b = -1 \leq 0 \Rightarrow 0$
- $(0, 1): 0 \leq 0 \Rightarrow 0$
- $(1, 0): 0 \leq 0 \Rightarrow 0$
- $(1, 1): 1 > 0 \Rightarrow 1$

x_1	x_2	y
0	0	0
0	1	0
1	0	0
1	1	1

Woche 9: (VL10 Lineare Modelle für Regression)

- Aufgabe 3 vom Kapitel "Lineare Modelle für Regression"
A: Siehe : Data Science 25, penguin Ordner, Ha_Woche7_Modellkomplexität_LineareRegression

Woche 10: (VL11 Lineare Modelle für Klassifikation)

- Aufgaben 1 und 2 vom Kapitel "Lineare Modelle für Klassifikation"

Aufgabe 1 - Logistische Regression interpretieren

Angenommen, wir haben einen Datensatz an Dokumenten, für welche wir folgende Features extrahiert haben und für welche eine logistische Regression mit folgenden Gewichten und Bias $b = 0.1$ trainiert worden ist, um das Sentiment des Textes vorherzusagen:

Feature	Feature Beschreibung	Gelerntes Gewicht
x_1	Zahl positiver Wörter im Dokument	2.5
x_2	Zahl negativer Wörter im Dokument	-5
x_3	1, falls "no" im Dokument, sonst 0	-1.2
x_4	Anzahl Pronomen in 1. und 2. Person	0.5
x_5	1, falls "!" im Dokument, sonst 0	2.0
x_6	$\log(\text{Anzahl an Wörtern im Dokument})$	0.7

Positive Wörter sind so was wie "great", "nice", etc.

a) Was lernen Sie aus den Gewichten über das Modell? Wie wichtig ist welches Feature und welchen Beitrag leistet es?

A: Es gilt:

Positives Gewicht: Feature erhöht Wahrscheinlichkeit für Klasse 1.

Negatives Gewicht: Feature verringert Wahrscheinlichkeit für Klasse 1.

Und je größer w_i desto größer der Einfluss auf die Variable/Vorhersage.

Das Feature x_1 steht für die Zahl positiver Wörter im Dokument und ist mit 2,5 gewichtet. Also das positive Gewicht erhöht die Wahrscheinlichkeit auf die Klasse1/Vorhersage.

Das Feature x_2 reduziert hingegen die Wahrscheinlichkeit auf die Klasse1/Vorhersage. Und x_2 ist mit -5 gewichtet. D.h. wenn negative Wörter im Dokument vorkommen dann ist deren Einfluss doppelt so stark gewichtet und verringern die Wahrscheinlichkeit von Klasse1. Die beiden Features sind sehr wichtig weil sie einen hohen Einfluss haben und x_2 ist am wichtigsten.

x_5 ist ebenfalls wichtig, weil es mit 2.0 gewichtet ist und wenn ein „!“ im Dokument ist dann erhöht das Feature x_5 die Wahrscheinlichkeit der Vorhersage.

Die anderen Features haben nicht so einen großen Einfluss und es gilt weiter das Prinzip siehe oben.

L Lösung: Letzte Aufgabe 1 (nicht sehr sortiert)

Am Vorzeichen sieht man, ob es positiv oder negativ

Anzahl an Wörtern ist logarithmisch → der Einfluss der Textmenge setzt sich nicht linear fort (unterschied zwischen 10 und 80 ist groß aber zwischen 10 und 1000 nicht mehr durch log)

Ein negatives Wort ist doppelt so stark wie ein positives Wort.

Sentiment: gute oder schlechte Stimmung in Dokument (auch feinere Schemata möglich: z.B. Vader Modell (psychologie: lvdv))

FRAGE: Wieso ist die Anzahl der Wörter logarithmisch?

- b) Berechnen Sie die Wahrscheinlichkeiten, dass das Sentiment für das Dokument in der unteren Abbildung als positiv ($P(y = 1)$) bzw. negativ ($P(y = -1)$) vorhergesagt wird. Leiten Sie daraus ab, zu welcher Klasse das Dokument zugewiesen werden würde.

A: Feature Vektor () x Gewichte Vektor () + 0.1 Bias

Positive und Negative Wörter als Matrixen aufzeichnen (x1-x6) → Feature Vektor () x Gewichtevektor () + 0.1 Bias = 0.73

Die Wahrscheinlichkeit, dass (y=1) ist = Sigmuidfunktion (0.73) = 0.68

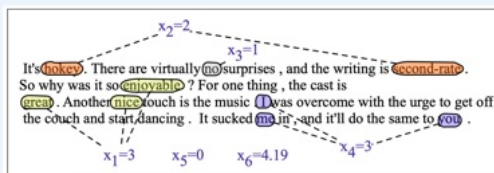
→ die Vorhersage ist 0.68, also 1 (Klasse)

Logistische Regression = clf

clf.predikt → Klasse

clf.predict_proba() → Wahrscheinlichkeit

- c) Berechnen Sie den Wert der Loss Function (Cross Entropy Loss) für das Beispiel aus der Abbildung, einmal unter der Annahme, dass der korrekte Wert der Output Variable $y = 1$ ist, und einmal unter der Annahme, $y = 0$. Überzeugen Sie sich von der Richtigkeit der grundlegenden Logik der Cross Entropy Loss für dieses Beispiel.



- c) Berechnen Sie den Wert der Loss Function (Cross Entropy Loss) für das Beispiel aus der Abbildung, einmal unter der Annahme, dass der korrekte Wert der Output Variable $y = 1$ ist, und einmal unter der Annahme, $y = 0$. Überzeugen Sie sich von der Richtigkeit der grundlegenden Logik der Cross Entropy Loss für dieses Beispiel.

A: Loss Function (Cross Entropy Loss) berechnen:

L Lösung:

Wenn der Wahre Wert 1 ist, ist der Fehler kleiner, als wenn der Wahre Wert 0 ist.

CEL (y (wahrer Wert), ydach (Vorhersage))

= -y*log(ydach) (fällt weg, wenn y=0) + (1-y)*log(1-ydach) (fällt weg bei y=1)

y=1 CEL(1, 0.86) = -1 * log(0.68) = 0.38

Aufgabe 2 - Logistische Regression implementieren

Trainieren Sie eine Logistische Regression anhand des Palmer Penguins Datensatzes. Vergleichen Sie die Ergebnisse mit den Ihres k -NN Modells. Welches performt besser?

noch machen!?

Eigene Fragen:

1) Wie berechne ich die Wahrscheinlichkeiten in Aufgabe 1b)?

Siehe rote Mappe

2) Ist die Loss Function die Cross Entropy Loss? Oder ist Cross Entropy Loss eine Unterkategorie von der Loss Function?

A: Ja

Aufgabe 1 - Cross validation mit Ridge und Lasso

Führen Sie eine Cross Validation der linearen Regression oder der logistischen Regression und berücksichtigen Sie dabei die drei Regularisierungsverfahren (Ridge, lasso und Elastic Net). Vergleichen Sie die Scores auf der Trainings- und Validierungsmenge. Führen Sie anschließend die Cross Validation auf z-score standardisierten Daten durch. Was fällt Ihnen auf?

Aufgabe 2 - Warum führt L1-Regularisierung zu Feature Selection?

Betrachten Sie ein Regressionsproblem mit zwei Koeffizienten w_1 und w_2 . Angenommen, die Loss Function MSE hat ihr Minimum bei $w_1 = 3, w_2 = 2$.

1. Betrachten Sie den 2-dimensionalen Raum der Parameter w_1 und w_2 . Die Loss Function hat einen konstanten Wert auf Ellipsen um den optimalen Gewichte-Punkt $(3, 2)$. Zeichnen Sie um den Punkt $(3, 2)$ herum 3–4 Ellipsen mit derselben Grundform. Das genaue Breite/Höhe-Verhältnis ist egal, sollte aber konstant bleiben.
2. Die Ridge-Regularisierung verwendet die L-2-Norm, d.h. dadurch wird dem Ausdruck $w_1^2 + w_2^2$ eine obere Grenze gesetzt. Die Ungleichung $w_1^2 + w_2^2 \leq t$ entspricht geometrisch einem Kreis. Zeichnen Sie in die bestehende Zeichnung einen Kreis für einen Radius Ihrer Wahl.
3. Zeichnen Sie nun die L1-Beschränkung (Lasso) $|w_1| + |w_2| \leq t$ ebenfalls ein. Geometrisch ergibt dies eine Raute mit Spitzen auf der Achse.
4. Verkleinern Sie den Wert t , bis die jeweilige Beschränkungsregion die innerste Ellipse gerade berührt. Markieren Sie jeweils den Berührungspunkt.

Welcher Regularisierer (L1 oder L2) berührt eine Ellipse häufiger an einem Punkt, an dem mindestens ein Koeffizient genau 0 ist (also auf den Achsen)? Erklären Sie in 2–3 Sätzen, warum dies zur Merkmalsselektion führt.

Aufgabe 2: richtig gemacht, Lösung in grüner folie

Aufgabe 1 — Confusion Matrix

Ein binärer Klassifikator wurde anhand von 1000 Testbeispielen evaluiert, in denen 50 % aller Beispiele negativ sind. Es wurde festgestellt, dass der Klassifikator einen Recall von 60 % und eine Genauigkeit von 70 % aufweist. Erstellen Sie die zugehörige Confusion Matrix. Berechnen Sie anschließend die Precision und den F1-Score des Klassifikators.

A: Siehe Notizen (confusion Matrix) (Lösung aus Übung)

Aufgabe 2 — Confusion Matrix

Beim Training eines probabilistischen Klassifikators werden Wahrscheinlichkeiten für die Zugehörigkeit zu den jeweiligen Klassen berechnet. Ein Datenpunkt wird üblicherweise der Klasse mit der höchsten Wahrscheinlichkeit zugeordnet. Eine Erweiterung wäre, einen sog. Cut-Off Wert festzulegen, welchen die Wahrscheinlichkeit zusätzlich überschreiten muss.

Die untere Tabelle zeigt die Wahrscheinlichkeiten für 8 Testdaten sowie die wahre Klasse. Für eine Zuweisung zu den Klassen 1 und 0 gilt folgende Regel: Ist die Wahrscheinlichkeit gleich oder größer als ein vorgegebener Cut-Off-Wert, ordne Klasse 1 voraus, andernfalls Klasse 0.

Berechnen Sie für die Cut-Off-Werte 0.8, 0.7 und 0.3 jeweils die Genauigkeit, Precision und den Recall des Klassifikators. (Sie dürfen die Aufgabe gerne mit einem kleinen Programm lösen.)

example true class probability		
x1	1	0.94
x2	1	0.75
x3	0	0.63
x4	0	0.56
x5	0	0.44
x6	1	0.25
x7	0	0.19
x8	0	0.13

Aufgabe 3 — Implementierung

Berechnen Sie mit Hilfe der `scikit-learn` Library die Confusion Matrix sowie den sog. Classification Report, welcher Precision, Recall und F1-Score enthält, für eines der bereits von Ihnen trainierten Klassifikationsmodelle.

Fragen der Woche 12: Tree Based Models

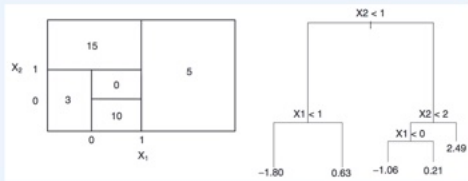
Aufgabe 1 - Partition des Feature-Raums

Konstruieren Sie ein Beispiel für einen Datensatz mit 2 Features und erstellen Sie eine Partition des Feature-Raums in 6 Rechtecke, welche durch die Anwendung eines Decision Tree Verfahrens (recursive binary split) entstanden sein konnte. Labeln Sie im Graphen die Rechtecke mit $R_1, \dots, R_6$ sowie die zugehörigen Schwellwerte.

Aufgabe 2 - Entscheidung im Ensemble

Sie haben ein Ensemble (Bagged Trees oder Random Forest) bestehend aus 10 Entscheidungsbäumen für Klassifikation eines Datensatzes mit 2 Klassen, grün und rot. Für einen Datenpunkt x liefern die 10 Bäume folgende Wahrscheinlichkeiten für die Zugehörigkeit zu Klasse rot: (0.1, 0.2, 0.15, 0.55, 0.6, 0.2, 0.75, 0.7, 0.6, 0.5). Es gibt nun 2 Verfahren, um die Ergebnisse der einzelnen Bäume zu kombinieren: Mehrheitsvotum sowie Mittelwert der Wahrscheinlichkeiten und danach die Zuordnung zur Klasse mit Wahrscheinlichkeit > 0.5 . Zu welchem Ergebnis würden die beiden Verfahren jeweils führen?

Aufgabe 3 - Von Feature-Partition zu Entscheidungsbaum



1. Skizzieren Sie einen Entscheidungsbaum für Regression basierend auf der Partition des Feature-Raums im Bild links. Die Zahlen in den Rechtecken sind die Mittelwerte der Zielvariable.
2. Skizzieren Sie eine Partition des Feature-Raums zum Entscheidungsbaum im Bild rechts.

Aufgabe 4 - Implementierung

1. Trainieren Sie einen Entscheidungsbaum auf dem Palmer Penguins Datensatz, mit vergleichbarer Auswahl von Features und Zielvariable wie in bisherigen Hausaufgaben. Die Baumtiefe darf maximal 4 betragen. Visualisieren Sie den trainierten Baum. Bonus: Machen Sie sich mit der Python-Bibliothek graphviz vertraut und erstellen Sie eine Visualisierung des Baums mit dieser Bibliothek.
2. Berechnen Sie verschiedene geeignete Fehlermetriken auf der Trainings- und der Testmenge.

implementierung muss ich noch machen

Fragen und Antworten:

1) Was ist eine Loss Function?

Bsp. bei der Linearen Regression ist der MSE eine Loss Function. Die lineare Regression wird auch OLS genannt. OLS bezeichnet ein konkretes Schätzverfahren für die Parameter der linearen Regression.

2) Nenne Annahmen der Linearen Regression:

- a. Linearität: Der bedingte Erwartungswert soll linear sein.
- b. Unabhängigkeit:
Die Fehlerterme sind unabhängig voneinander und haben dieselbe Verteilung.
- c. Homoskedastizität:
Die Varianz des Rauschens ist konstant.
- d. Normalverteiltes Rauschen:
Die Fehlerterme sind (für statistische Tests) normalverteilt.
(Für reine Vorhersagen ist diese Annahme nicht notwendig).

3) Was bedeutet Heteroskedastizität?

A: Bei Heteroskedastizität ist die Streuung der Fehler nicht konstant. Die Varianz des Fehlers hängt also vom Wert der Features ab. (sieht aus wie ein Trichter)

4) Was passiert wenn die 4 Annahmen (Linearität, Unabhängigkeit, Homoskedastizität, Normalverteiltes Rauschen), nicht gelten?

1. Linearitätsannahme verletzt

- Die Beziehung zwischen Features und Zielvariable ist nicht linear
- Das Modell ist systematisch verzerrt (Bias)
- Vorhersagen sind noch brauchbar, aber suboptimal.
- Lösungsmöglichkeiten: Feature Engineering, nichtlineare Modelle

2. Nicht -i.i.d. Daten (die Daten sind nicht independent, identically distributed (unabhängig und identisch verteilt, also jedes Datenbeispiel ist unabhängig von den anderen und alle Datenbeispiele stammen aus der selben Wahrscheinlichkeitsverteilung. Sie haben denselben Erwartungswert, dieselbe Varianz etc.)

Typisches Beispiel: Zeitreihen oder räumlich abhängige Daten.

- Folge: Die Unsicherheitsschätzungen sind falsch
- Vorhersage kann trotzdem funktionieren, aber man muss vorsichtig sein.
- Lösungen:
 - Zeitreihenmodelle
 - Cross Validation mit zeitlicher Struktur.

5) Wie können lineare Modelle für Klassifikation angepasst werden?

- A: mit Sigmoid-Funktion (der Output wird normalisiert) und Softmax (bei mehr als 2 Klassen muss die Sigmoid Funktion verallgemeinert werden.. auch Normalisierung.)

6) Was ist die Cross Entropy Loss?

A: Sie wird als Loss Function für Klassifikation eingesetzt. Z.B. für die Logistische Regression, ein lineares Klassifikationsmodell.

Bei der logistischen Regression bestraft die Cross Entropy Loss falsche Vorhersage-Wahrscheinlichkeiten und maximiert die Wahrscheinlichkeit der korrekten Klasse.

Antworten von der VL 13 Regularisierung iPad:

7) Wie wirkt sich Regularisierung auf Modellkomplexität aus?

Ridge (L2-Regularisierung) und Lasso (L1-Regularisierung) sowie Elastic Net sind Regularisierungsverfahren, die die Modellkomplexität beeinflussen.

Bei Ridge werden Gewichte verkleinert und die Komplexität leicht reduziert.

Bei Lasso werden Gewichte auch eliminiert und die Komplexität stark reduziert.
Bei Elastic Net werden Ridge und Lasso kombiniert und die Komplexität flexibel kontrolliert.

Was gibt es für Regularisierungsverfahren für lineare und logistische Regression?

Die zentralen Regularisierungsverfahren für lineare und logistische Regression sind Lasso (L1), Ridge (L2) und Elastic Net. Die lineare und die logistische Regression unterscheiden sich in der verwendeten Loss-Funktion: Bei der linearen Regression wird typischerweise der MSE verwendet, bei der logistischen Regression der Cross-Entropy-Loss.

8) Wie ist der Unterschied in ihrer Wirkweise der Regularisierungsverfahren von der Linearen Regression und der logistischen Regression?

A: Ridge bestraft große Gewichte.

Wirkung: Die Gewichte werden kleiner, aber meist nicht exakt 0. Alle Features bleiben im Modell.

Einfluss auf Modellkomplexität: Das Modell wird glatter und weniger flexibel. Die Komplexität wird kontrolliert aber nicht stark reduziert.

Lasso bestraft die Summe der Beträge der Gewichte.

Wirkung: Viele Gewichte werden exakt 0.

Es gibt eine automatische Feature Selektion.

Einfluss auf Modellkomplexität: Modell wird deutlich einfacher

Weniger aktive Features -> geringe Komplexität

Elastic Net ist eine Kombi aus Ridge und Lasso.

Wirkung: -Feature Selektion von (L1 mit Lasso)

-Stabilität bei korrelierten Features (L2 mit Ridge)

Einfluss auf Modellkomplexität:

Flexibel steuerbare Reduktion der Komplexität

Die Regularisierungsverfahren Lasso (L1), Ridge (L2) und Elastic Net wirken bei linearer und logistischer Regression grundsätzlich gleich: Sie bestrafen große oder viele Gewichte und reduzieren dadurch die Modellkomplexität. Ridge verkleinert Gewichte, Lasso setzt Gewichte auf 0 und ermöglicht Feature-Selektion, Elastic Net kombiniert beide Effekte. **Der Unterschied zwischen linearer und logistischer Regression liegt nicht in der Regularisierung, sondern in der verwendeten Loss-Funktion:** Lineare Regression verwendet typischerweise den MSE, logistische Regression den Cross-Entropy-Loss.

9) Warum wird die varianz-basierte Standardisierung mittels z-score im Zusammenhang mit Regularisierung benötigt?

A: Regularisierungsverfahren wie Ridge (L2) und Lasso (L1) bestrafen Gewichte w_i . Die Größe der Gewichte hängt stark auf der Skala der Features. Bsp. bei verschiedenen Skalen wie Alter und Einkommen, da würde Alter stärker bestraft werden und Einkommen weniger bestraft werden. Mit der Standardisierung bekommen alle Features den Mittelwert 0 und die Standardabweichung 1. Die Gewichte werden vergleichbar und die Regularisierung wirkt fair auf alle Features.

10) Ist die 5-fold-cross-validation dafür da um das beste α auszuwählen?

Ja, es kann so das beste α gefunden werden, das den Kompromiss zwischen Underfitting und Overfitting optimiert.